ZERO DOWNTIME DATABASE CHANGES

Schema Migrations, Data Type Changes & Best Practices

Expand & Contract • gh-ost • Feature Flags • CDC

Production-Grade Approaches for Live Databases

Interview & System Design Preparation

TABLE OF CONTENTS

1. The Problem: Why Schema Changes Break Production

2. Which ALTER TABLE Operations Lock the Table?

3. Approach 1: Expand and Contract Pattern

4. Approach 2: Online Schema Migration Tools (gh-ost)

5. Approach 3: Blue-Green Database Migration

6. Approach 4: Feature Flags + Backward Compatible Changes

7. Approach 5: Views as Abstraction Layer

8. Approach 6: CDC (Change Data Capture)

9. Approach 7: Flyway / Liquibase (Migration Versioning)

10. Deep Dive: Changing Data Types

11. Edge Cases: NOT NULL, Indexes, Foreign Keys

12. Corrected Deployment Timeline

13. Approach Comparison

14. Interview-Ready Answers

1. The Problem

When you change database schema in production (add/remove/rename columns, change data types), your app is live with users. During deployment, both old and new code versions may be running simultaneously. Running ALTER TABLE directly can lock the table or break the old code.

```
Scenario: Rename column 'name' to 'full_name'

  ALTER TABLE users RENAME COLUMN name TO full_name;
  -> Old code: SELECT name FROM users  -> FAILS (column gone)
  -> New code: SELECT full_name FROM users -> WORKS

During rolling deployment, BOTH versions are running.
Old code crashes instantly.
```

Rule: Every database migration must be backward compatible. Old code must continue to work after the migration runs. Never remove or rename something the current code depends on.

2. Which Operations Lock the Table?

Not all ALTER TABLE operations lock the table. It depends on the database engine and version. Understanding this determines whether you can run ALTER directly or need tools like gh-ost.

Operation	MySQL 8+	PostgreSQL 11+	Oracle 12c+
ADD COLUMN (nullable, no default)	INSTANT ■	No lock ■	No lock ■
ADD COLUMN with DEFAULT	LOCKS ■	No lock ■	No lock ■
DROP COLUMN	LOCKS ■	No lock ■	No lock ■
RENAME COLUMN	INSTANT ■	No lock ■	No lock ■
CHANGE DATA TYPE	LOCKS ■	LOCKS ■	LOCKS ■
ADD INDEX	ONLINE ■	CONCURRENT ■	ONLINE ■
ADD NOT NULL	LOCKS ■	LOCKS ■	LOCKS ■

MySQL Algorithms

```
-- Try INSTANT first (milliseconds, no lock)
ALTER TABLE users ADD COLUMN age INT, ALGORITHM=INSTANT;

-- If INSTANT not supported, try INPLACE
ALTER TABLE users ADD COLUMN age INT, ALGORITHM=INPLACE, LOCK=NONE;
-- Allows concurrent reads/writes

-- If neither works, MySQL falls back to COPY -> LOCKS table
-- In that case, use gh-ost instead
```

PostgreSQL Safe ADD COLUMN with Default

```
-- DON'T (locks in PG < 11):
ALTER TABLE users ADD COLUMN age INT DEFAULT 0;

-- DO (two steps, no lock):
ALTER TABLE users ADD COLUMN age INT;             -- no lock
ALTER TABLE users ALTER COLUMN age SET DEFAULT 0;  -- no lock
-- Then backfill existing rows in batches
```

3. Approach 1: Expand and Contract Pattern

MOST COMMON — Add new column alongside old one. Migrate data gradually. Remove old one later. Three phases: EXPAND → MIGRATE → CONTRACT.

Example: Rename 'name' to 'full_name'

```
WRONG (causes downtime):
  ALTER TABLE users RENAME COLUMN name TO full_name;
  -> Old code breaks instantly

RIGHT (zero downtime):

Phase 1 - EXPAND:
  ALTER TABLE users ADD COLUMN full_name VARCHAR(255);
  Old code still works (uses 'name'). New column empty.

Phase 2 - MIGRATE:
  Deploy dual-write code (writes to BOTH columns)
  Backfill: UPDATE users SET full_name = name (in batches)

Phase 3 - SHIFT:
  Deploy code that READS from 'full_name'
  Still writes to both (safety net)

Phase 4 - CONTRACT:
  Deploy code that ONLY uses 'full_name'
  ALTER TABLE users DROP COLUMN name;
```

Table State at Each Phase

```
Phase 1:  | name    | full_name |
          | 'John'  | NULL      |   <- new column added

Phase 2:  | name    | full_name |
          | 'John'  | 'John'    |   <- data copied, dual write

Phase 4:  | full_name |
          | 'John'    |             <- old column removed
```

4. Approach 2: Online Schema Migration Tools

FOR LARGE TABLES — Tools like gh-ost and pt-online-schema-change perform ALTER TABLE without locking, regardless of operation type.

How gh-ost Works

```
Step 1: Create GHOST table with NEW schema
  users (original)         _users_gho (ghost)
  | id, name  |            | id, name, age |  <- new schema
  | 100M rows |            | 0 rows        |

Step 2: Copy data in small CHUNKS
  Copy rows 1-1000, then 1001-2000...
  Original table fully usable during copy

Step 3: Capture live changes via binlog
  Any INSERT/UPDATE/DELETE on original
  -> Also applied to ghost table

Step 4: Atomic RENAME when caught up
  RENAME TABLE users TO _users_old,
               _users_gho TO users;
  Instant swap (microseconds)

Step 5: DROP TABLE _users_old;
```

Tools by Database

Database	Tool	How It Works
MySQL	gh-ost (GitHub)	Binlog-based, ghost table swap
MySQL	pt-online-schema-change (Percona)	Trigger-based, ghost table swap
PostgreSQL	pg_repack	Repacks table without lock
PostgreSQL	pgroll	Versioned schema migrations
Any	Flyway / Liquibase	Migration versioning (not lock-free)

5. Approach 3: Blue-Green Database Migration

Run two databases simultaneously. Blue is current and live. Green has the new schema. Keep both in sync via replication. Switch traffic to Green when ready. Easy rollback by switching back to Blue.

```
Blue DB (live)              Green DB (new schema)
| users        |   sync     | users        |
| - name       | ---------> | - full_name  |
| - email      |            | - email      |

Step 1: Set up Green with new schema
Step 2: Replicate + transform data continuously
Step 3: Deploy new code pointing to Green
Step 4: Switch traffic -> Green is primary
Step 5: Verify -> decommission Blue
```

Pros	Cons
Easy rollback (switch back to Blue)	Double database resources temporarily
Can test Green thoroughly	Complex data sync during transition
Clean cut-over	Not suitable for very large databases

6. Approach 4: Feature Flags

Deploy code that works with both old and new schema. Use feature flags to toggle between them. Gradually roll out from 10% to 100% of users.

```
// Java - Spring Boot example
public String getUserName(Long id) {
    User user = userRepo.findById(id);
    if (featureFlags.isEnabled('USE_FULL_NAME')) {
        return user.getFullName();    // new column
    }
    return user.getName();            // old column
}

Rollout: flag OFF -> 10% -> 50% -> 100% -> remove flag
```

7. Approach 5: Views as Abstraction Layer

Put a VIEW between your app and the table. Change the underlying table, update the view to map new columns to old names. App never notices.

```
-- App reads from VIEW, not table directly
CREATE VIEW users_view AS
SELECT name, email FROM users;

-- Rename column in actual table
ALTER TABLE users RENAME COLUMN name TO full_name;

-- Update VIEW to map new to old
CREATE OR REPLACE VIEW users_view AS
SELECT full_name AS name, email FROM users;

-- App still sees 'name' - nothing breaks
```

8. Approach 6: CDC (Change Data Capture)

Capture all changes from the old table as events, transform them, and replay into a new table with the new schema. Switch when caught up. Best for cross-database migrations.

```
Step 1: Enable CDC (Debezium / AWS DMS)
  Every INSERT/UPDATE/DELETE captured as event

Step 2: Create new table with new schema

Step 3: CDC transforms and writes to new table
  Old event: { 'name': 'John' }
  Transform: { 'full_name': 'John' }

Step 4: When caught up -> switch app to new table

Tools: Debezium + Kafka, AWS DMS, AWS Glue
```

9. Approach 7: Flyway / Liquibase

Version your database changes like Git versions code. Each migration file runs in order, only once. The tool tracks which migrations have been applied.

```
src/main/resources/db/migration/
  V1__create_users_table.sql
  V2__add_full_name_column.sql
  V3__backfill_full_name.sql
  V4__drop_name_column.sql

-- V2__add_full_name_column.sql
ALTER TABLE users ADD COLUMN full_name VARCHAR(255);

-- V3__backfill_full_name.sql (run in batches in production)
UPDATE users SET full_name = name WHERE full_name IS NULL;

-- V4__drop_name_column.sql (only after code stops using 'name')
ALTER TABLE users DROP COLUMN name;
```

10. Deep Dive: Changing Data Types

MOST COMPLEX CHANGE — Changing a data type (e.g., INT to DECIMAL) locks the table in ALL databases because every row must be rewritten. The Expand and Contract pattern is required.

Example: Change 'price' from INT to DECIMAL(10,2)

Phase 1: ADD new column with new type

```
-- PostgreSQL (no lock, nullable, no default)
ALTER TABLE products ADD COLUMN price_new DECIMAL(10,2);

-- MySQL 8+ (instant if possible)
ALTER TABLE products ADD COLUMN price_new DECIMAL(10,2), ALGORITHM=INSTANT;

-- MySQL (if INSTANT not supported) -> use gh-ost
gh-ost --alter='ADD COLUMN price_new DECIMAL(10,2)' --execute

-- Table now:
-- | id | name   | price (INT) | price_new (DECIMAL) |
-- | 1  | Laptop | 999         | NULL                |
```

Phase 2: Deploy DUAL WRITE code

```
// Java - write to BOTH columns
@PrePersist @PreUpdate
public void syncColumns() {
    if (priceNew != null) {
        price = priceNew.intValue();
    }
    if (price != null && priceNew == null) {
        priceNew = BigDecimal.valueOf(price);
    }
}
```

Phase 3: BACKFILL existing data in batches

```
-- DON'T (locks table):
UPDATE products SET price_new = CAST(price AS DECIMAL(10,2));

-- DO (batched, no lock):
UPDATE products
SET price_new = CAST(price AS DECIMAL(10,2))
WHERE price_new IS NULL
LIMIT 1000;
-- Sleep 100ms between batches
-- Repeat until 0 rows affected
```

Phase 4: SHIFT reads to new column

```
// Feature flag rollout
if (featureFlags.isEnabled('USE_DECIMAL_PRICE')) {
    return product.getPriceNew();      // new DECIMAL
} else {
    return BigDecimal.valueOf(product.getPrice());  // old INT
}

// Rollout: 10% -> 50% -> 100%
```

Phase 5: VERIFY data integrity

```
-- Both columns match?
SELECT COUNT(*) FROM products
WHERE price_new != CAST(price AS DECIMAL(10,2));
-- Should return 0

-- Any NULLs remaining?
SELECT COUNT(*) FROM products WHERE price_new IS NULL;
-- Should return 0
```

Phase 6: STOP writing to old column

```
// Remove dual write, only use price_new
public void updatePrice(Long id, BigDecimal newPrice) {
    Product p = repo.findById(id);
    p.setPriceNew(newPrice);    // only new column
    repo.save(p);
}
```

Phase 7: DROP old column

```
-- PostgreSQL (no lock)
ALTER TABLE products DROP COLUMN price;

-- MySQL (use gh-ost for large tables)
gh-ost --alter='DROP COLUMN price' --execute
```

Phase 8: RENAME new column (optional)

```
-- PostgreSQL (no lock)
ALTER TABLE products RENAME COLUMN price_new TO price;

-- MySQL 8+ (instant)
ALTER TABLE products RENAME COLUMN price_new TO price, ALGORITHM=INSTANT;
```

Final State

```
BEFORE:  | id | name   | price (INT) |
         | 1  | Laptop | 999         |

AFTER:   | id | name   | price (DECIMAL) |
         | 1  | Laptop | 999.00          |

Zero downtime. Zero data loss.
```

11. Edge Cases

NOT NULL Constraints

```
-- DON'T (full table scan, locks):
ALTER TABLE products ALTER COLUMN price_new SET NOT NULL;

-- DO (PostgreSQL 12+, non-blocking):

-- Step 1: Add CHECK constraint (no lock)
ALTER TABLE products ADD CONSTRAINT price_not_null
CHECK (price_new IS NOT NULL) NOT VALID;

-- Step 2: Validate (scans but doesn't lock writes)
ALTER TABLE products VALIDATE CONSTRAINT price_not_null;

-- Step 3: Now safe to set NOT NULL (no scan needed)
ALTER TABLE products ALTER COLUMN price_new SET NOT NULL;
```

Indexes

```
-- Create index on new column WITHOUT locking:

-- PostgreSQL
CREATE INDEX CONCURRENTLY idx_price_new ON products(price_new);

-- MySQL
ALTER TABLE products ADD INDEX idx_price_new(price_new),
ALGORITHM=INPLACE, LOCK=NONE;

-- After old column dropped, drop old index
DROP INDEX idx_products_price;
```

Foreign Keys

```
Order matters - can't drop column with active FK

Step 1: Add FK on new column (pointing to same ref)
Step 2: Migrate data
Step 3: Drop FK on old column
Step 4: Drop old column
```

Data That Doesn't Fit New Type

```
-- Check BEFORE migration:
SELECT id, price FROM products
WHERE price > 99999999;  -- exceeds DECIMAL(10,2) max

-- Handle in transformation:
UPDATE products SET price_new = CASE
    WHEN price > 99999999 THEN 99999999.99
    ELSE CAST(price AS DECIMAL(10,2))
END
WHERE price_new IS NULL LIMIT 1000;
```

12. Corrected Deployment Timeline

Key insight: Day 1 must use a non-locking method. PostgreSQL ADD COLUMN (nullable, no default) is instant. MySQL uses ALGORITHM=INSTANT if supported, otherwise gh-ost. Never run a locking ALTER on a production table.

```
Day 1: ADD new column (NO LOCK method)
  PostgreSQL: ALTER TABLE ... ADD COLUMN ... ;         -- instant
  MySQL 8+:   ALTER TABLE ... ADD COLUMN, ALGORITHM=INSTANT;
  MySQL (fallback): gh-ost --alter='ADD COLUMN ...'

Day 2: Deploy dual-write code
  Writes to BOTH old and new columns
  Reads from old column

Day 3: Backfill in BATCHES of 1000
  UPDATE ... SET new_col = old_col WHERE new_col IS NULL LIMIT 1000
  Sleep 100ms between batches

Day 4: Verify data integrity (SQL checks)

Day 5: Deploy read-from-new code
  Feature flag: 10% -> 50% -> 100%

Day 6: Monitor for errors

Day 7: Deploy remove-old-write code
  Stop writing to old column

Day 8: DROP old column
  PostgreSQL: ALTER TABLE ... DROP COLUMN;  -- no lock
  MySQL: gh-ost --alter='DROP COLUMN ...'

Day 9: RENAME new column to original name
  No lock in both PG and MySQL 8+
```

13. Approach Comparison

Approach	Complexity	Downtime	Best For
Expand & Contract	Low	Zero	Most changes ■
Online Schema Tools	Medium	Zero	Large tables ■
Blue-Green DB	High	Seconds	Major overhauls
Feature Flags	Medium	Zero	Gradual rollout
Views Abstraction	Low	Zero	Column renames
CDC	High	Zero	Cross-DB migration
Flyway / Liquibase	Low	Zero	Migration versioning

What Companies Actually Combine

```
1. FLYWAY / LIQUIBASE  -> version all migrations
2. EXPAND & CONTRACT   -> backward compatible SQL
3. gh-ost / pt-osc     -> large table alterations
4. FEATURE FLAGS       -> gradual rollout
5. CDC                 -> cross-database migrations
```

Quick Decision: Which Method?

```
Is the operation non-locking natively?
  |
  +-- YES (ADD nullable col in PG, INSTANT in MySQL 8+)
  |   -> Run ALTER TABLE directly
  |
  +-- NO (change type, add default in old MySQL, NOT NULL)
      |
      +-- Table < 1M rows?
      |   -> Short maintenance window OR gh-ost
      |
      +-- Table > 1M rows?
          -> MUST use gh-ost / pt-online-schema-change
```

14. Interview-Ready Answers

Q: How do you deploy database changes without downtime?

"I use the Expand and Contract pattern. Add the new column first, deploy dual-write code, backfill existing data in batches, shift reads to the new column using feature flags, then drop the old one. Flyway manages migration versioning. Every step is backward compatible."

Q: Does ALTER TABLE always lock the table?

"No. In PostgreSQL, ADD COLUMN without a default is instant metadata-only. MySQL 8+ supports ALGORITHM=INSTANT for adding columns. But changing data types locks in all databases because every row must be rewritten. For locking operations, I use gh-ost which creates a shadow copy and swaps atomically with zero lock."

Q: How do you change a data type without downtime?

"I add a new column with the target type, deploy code that writes to both old and new columns, backfill in batches of 1000 rows with pauses, verify data integrity, shift reads via feature flags, stop writing to the old column, drop it, then rename the new column. NOT NULL constraints are added using NOT VALID CHECK first, then validated separately to avoid locking."

Q: How do you handle large table migrations?

"For tables over 1 million rows, I never run locking ALTER TABLE directly. I use gh-ost or pt-online-schema-change which create a ghost table with the new schema, copy data in chunks, capture live changes via binlog, and perform an atomic rename when caught up. Backfills are batched with 1000 rows per iteration and 100ms pauses to avoid load spikes."

Q: What tools do you use for database migrations?

"Flyway or Liquibase for versioning migrations. gh-ost for MySQL schema changes on large tables. CREATE INDEX CONCURRENTLY for PostgreSQL indexes. Feature flags for gradual code rollout. Debezium with Kafka for CDC-based cross-database migrations."